

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO5 ASSESSMENT TOOL 2 – Seminar on Emerging Technologies

Course Code:	CSA1008	Course Title:	Software Engineering
CO Assessed:	CO5	Assessment:	Assessment Tool 2 – Seminar on Emerging Technologies
Weightage:	20%	Total Marks:	25
CO5 Statement:	Evaluate emerging technologies and societal impacts, maintaining and evolving software systems responsibly		
Student Name:	B. Pushparaj	Reg. No.:	192421136
Date:	26.08.2026	Duration:	60 minutes

Code of Conduct

I **B. Pushparaj(192421136)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Annexure A – Seminar on Emerging Technologies

Instructions to Students:

Students must select the topic **strictly according to the Serial Number (S.No.) order** assigned to them.

S.No.	Seminar Topic
1	Generative AI and Large Language Models (LLMs)
2	AI Agents and Autonomous AI Systems
3	Multimodal Artificial Intelligence

4	Explainable and Trustworthy AI
5	Edge AI and Intelligent Edge Computing
6	Federated Learning and Privacy-Preserving AI
7	Deep Learning for Computer Vision
8	AI-Powered Medical Image Analysis
9	Natural Language Processing with Transformer Models
10	Retrieval-Augmented Generation (RAG)
11	AI in Cybersecurity and Threat Detection
12	Zero Trust Architecture and Modern Cybersecurity
13	Post-Quantum Cryptography
14	Quantum Computing and Quantum Algorithms
15	Quantum Machine Learning
16	Cloud-Native Computing and Microservices
17	Serverless Computing and Function-as-a-Service
18	Edge Computing and Fog Computing
19	Cloud Security and Confidential Computing
20	Digital Twins and Intelligent Simulation
21	Internet of Things (IoT) and Smart Systems
22	Industrial Internet of Things (IIoT)
23	5G/6G Networks and Next-Generation Communication
24	Blockchain Technology and Decentralized Applications
25	Web3 and Decentralized Internet
26	Smart Contracts and Blockchain Applications
27	Augmented Reality (AR) and Virtual Reality (VR)
28	Extended Reality (XR) and Immersive Technologies
29	Robotics and Autonomous Robots
30	Human–Robot Interaction and Collaborative Robots
31	Autonomous Vehicles and Intelligent Transportation Systems

32	Smart Cities and AI-Driven Urban Technologies
33	Metaverse Technologies and Virtual Worlds
34	Brain–Computer Interfaces (BCI)
35	Wearable Computing and Smart Wearable Devices
36	Green Computing and Sustainable IT
37	AI for Software Engineering and Code Generation
38	DevSecOps and Intelligent CI/CD Pipelines
39	Digital Forensics and AI-Based Cybercrime Investigation
40	Privacy-Enhancing Technologies and Data Protection

Seminar Presentation Structure:

1. Introduction to the Emerging Technology
2. Need and Motivation
3. Working Principle / Architecture
4. Key Components and Technologies
5. Applications / Real-World Use Cases
6. Advantages and Limitations
7. Recent Developments and Trends
8. Challenges and Future Scope
9. Case Study / Example
10. Conclusion and References

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
----------------------------	----------------------------	-----------------------	-------------------------------	-----------------------------------

Technical Content & Subject Knowledge (25%)	Comprehensive and accurate content; demonstrates excellent understanding of the emerging technology	Good technical understanding with minor gaps	Basic understanding with limited technical depth	Incomplete content or major technical errors
Organization & Presentation Structure (15%)	Excellent logical flow with clear introduction, content, conclusion, and effective transitions	Well-structured with minor organizational issues	Adequately structured but some sections lack clarity	Poorly structured and difficult to follow
Examples, Diagrams & Case Study (20%)	Excellent use of relevant diagrams, examples, applications, and real-world case studies	Good use of relevant examples and diagrams	Limited examples/diagrams with basic explanation	Inappropriate or insufficient examples/diagrams
Communication & Presentation Skills (20%)	Very clear, confident, engaging delivery with excellent eye contact, voice modulation, and time management	Clear and confident presentation with minor issues	Understandable but lacks confidence or engagement	Unclear delivery, excessive reading, or poor time management
Question Handling & Overall Performance (20%)	Answers questions accurately and confidently; demonstrates strong overall knowledge	Answers most questions correctly	Answers basic questions with some difficulty	Unable to answer questions or demonstrate adequate understanding

Criteria	Mark
Technical Content & Subject Knowledge (25%)	/5
Organization & Presentation Structure (15%)	/5
Examples, Diagrams & Case Study (20%)	/5
Communication & Presentation Skills (20%)	/5
Question Handling & Overall Performance (20%)	/5
TOTAL	/25

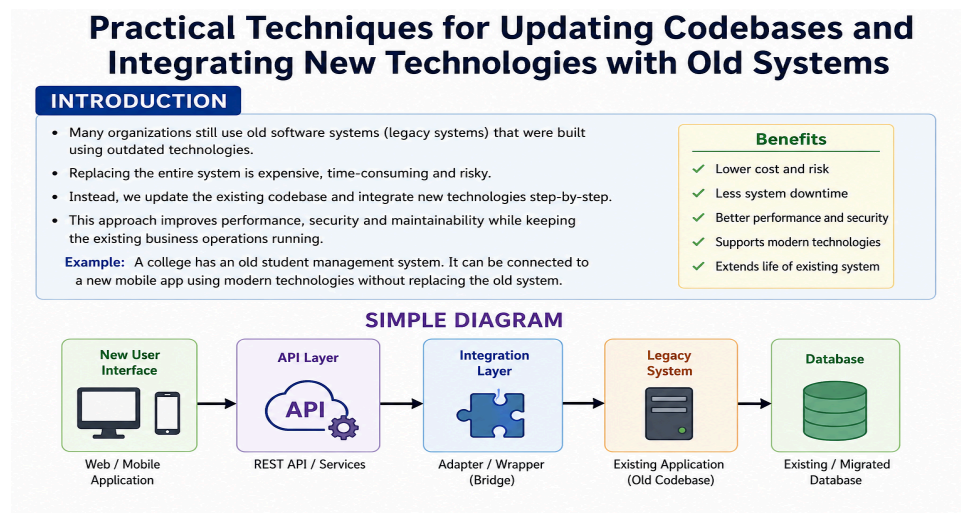
SEMINAR REPORT

Practical Techniques for Updating Codebases and Integrating New Technologies with Old Systems

1. Introduction

Many organizations still depend on old software systems called **legacy systems**. These systems may contain important business logic and valuable data, but they often use outdated technologies.

Completely replacing a legacy system can be expensive, risky, and time-consuming. Therefore, **software modernization** provides a practical solution by gradually improving the existing system and integrating it with modern technologies.



The main techniques include:

- Code refactoring
- API integration
- Adapter/Wrapper technique
- Database migration
- Automated testing
- Git and CI/CD
- Cloud and modern application integration

Your PPT describes modernization as the process of gradually updating existing software while keeping important functionality available.

2. What is a Legacy System?

A **legacy system** is an older software system that is still actively used by an organization.

It may contain:

- Old programming languages
- Outdated technologies
- Old database systems
- Complex source code
- Important business rules
- Large amounts of existing data

Although the technology may be old, the system can still be very important to the organization.

Example

A college may have an old desktop-based system for:

- Student registration
- Attendance
- Marks
- Examination
- Reports

Instead of completely replacing it, the college can connect a modern web or mobile application to the existing system.

Your PPT similarly identifies a legacy system as an old but still-used system containing important data and business logic.

3. Need and Motivation for Modernization

Software modernization is needed because old systems may become difficult to maintain and integrate with new technologies.

Major reasons

1. **Improve performance**
 - Modern technologies can improve system speed.
2. **Increase security**
 - Old systems may contain security weaknesses.
3. **Support new technologies**
 - Modern web, mobile, cloud and API technologies can be integrated.

4. Easy maintenance

- Refactoring makes complicated code easier to understand.

5. Reduce operating cost

- Modern infrastructure can reduce maintenance expenses.

6. Modern user interface

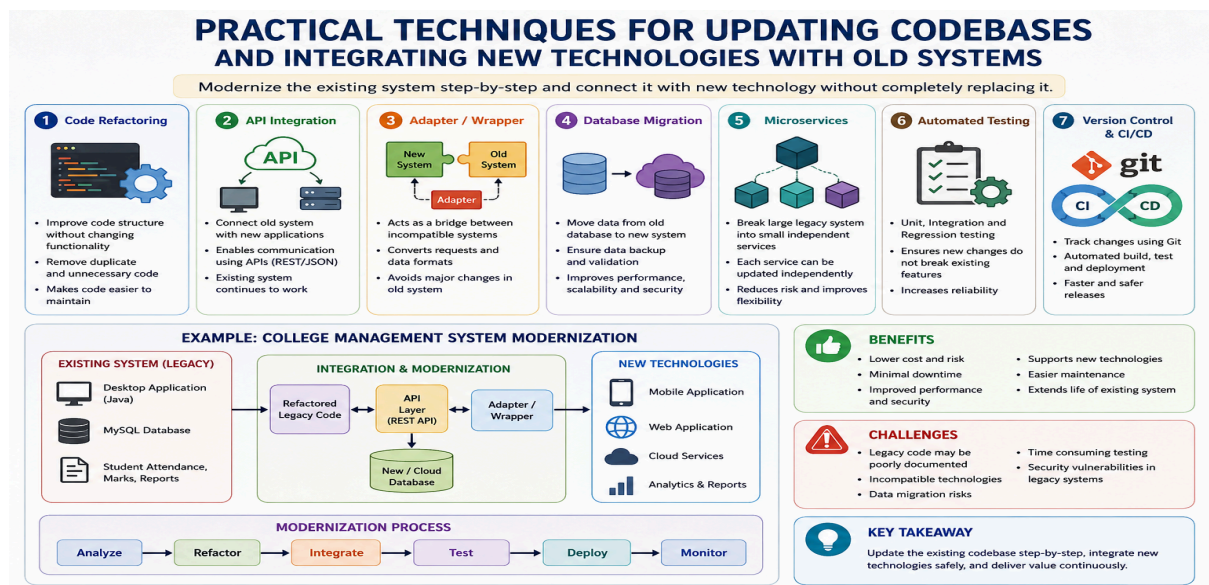
- Old desktop applications can be connected to modern web/mobile interfaces.

Your PPT gives the example of adding a mobile application to an existing desktop-based college system.

4. Working Principle / Architecture

The modernization approach connects the **old system** with **new technologies** instead of immediately replacing the old system.

Architecture



User → Modern Application → API → Adapter → Legacy System → Database

This allows the organization to introduce modern features without immediately removing the existing system.

5. Key Techniques

5.1 Code Refactoring

Refactoring means improving the internal structure of existing code without changing its original functionality.

Benefits

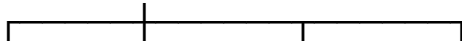
- Removes duplicate code
- Improves readability
- Makes maintenance easier
- Reduces complexity
- Makes future development easier

Example

Old Large Program



Refactoring



Login Attendance Marks Reports

Your PPT specifically describes refactoring as improving existing code while maintaining its functionality.

6. API Integration

An **API (Application Programming Interface)** allows different software applications to communicate.

Modern applications can communicate with old systems through APIs.

Example

Mobile Application



REST API



Legacy College System



Database

Advantages

- Connects old and new applications
- Allows mobile/web access

- Reduces need to modify legacy software
- Supports gradual modernization

Your PPT identifies REST APIs as a common method for connecting modern applications with existing systems.

7. Adapter / Wrapper Technique

An **Adapter or Wrapper** acts as a bridge between modern and legacy components.

It converts modern requests into a format that the old system understands.

Architecture

New Application

|



Adapter / Wrapper

|



Legacy Application

Example

A new mobile application sends a JSON request, but the old system expects a different format. The adapter converts the request before sending it to the legacy application.

This technique is useful when the old software cannot easily be modified.

8. Database Migration

Database migration involves moving information from an old database to a modern database.

Process

Old Database

|



Backup Data

|



Data Migration

|



Data Validation

|



Cloud / Modern Database

Important activities

- Backup existing data
- Clean unnecessary data
- Convert data formats
- Transfer data
- Validate migrated data
- Test the new database

Your PPT highlights backup and validation as important parts of database migration.

9. Automated Testing

Testing is very important when modifying a legacy system because existing functionality must continue working.

Types of testing

Testing	Purpose
Unit Testing	Tests individual components
Integration Testing	Tests communication between systems
Regression Testing	Ensures old features still work
API Testing	Tests communication through APIs
Performance Testing	Checks system performance
Security Testing	Identifies security problems

Example

After adding an online attendance feature, we should verify:

Login



Attendance



Marks



Reports

The original PPT specifically mentions unit, integration and regression testing for modernization projects.

10. Version Control and CI/CD

Git helps developers track changes to source code.

Developers can create separate branches for different modernization tasks and merge completed changes safely.

CI/CD Process

Developer



Git



Code Review



Automated Test



Build



Deploy

Benefits

- Tracks code changes
- Supports teamwork
- Reduces deployment errors
- Enables automated testing
- Makes rollback easier

Your PPT summarizes the process as **Code → Git → Test → Build → Deploy**.

11. Applications / Real-World Use Cases

Software modernization can be used in many areas.

1. Education

Old college management systems can be connected to:

- Mobile applications
- Web portals
- Online attendance
- Student dashboards

2. Banking

Legacy banking applications can be connected to modern:

- Mobile banking
- Online payment systems
- REST APIs

3. Healthcare

Existing hospital systems can be integrated with:

- Patient portals
- Appointment systems
- Modern reporting applications

4. Government

Old government databases can be connected with modern citizen-service portals.

5. Business

Old enterprise applications can be integrated with cloud services and modern dashboards.

12. Advantages

Major benefits

- Extends the life of existing software
- Reduces development cost
- Improves security
- Improves performance

- Supports modern applications
- Reduces downtime
- Makes future upgrades easier
- Protects existing business data

These benefits are also identified in your PPT.

13. Limitations

Despite its benefits, modernization has some challenges.

1. **Complex legacy code**
 - Old code may be difficult to understand.
2. **Compatibility issues**
 - New and old technologies may use different formats.
3. **Data migration risks**
 - Data may be lost or corrupted if migration is not properly managed.
4. **High initial effort**
 - Modernization requires planning and development time.
5. **Security concerns**
 - Connecting an old system to the internet can introduce security risks.
6. **Testing complexity**
 - Existing functionality must be carefully tested after every change.

14. Recent Developments and Trends

Modern software modernization increasingly uses:

- Cloud migration
- REST APIs
- Microservices
- Containerization
- Docker
- Automated testing
- Git-based development

- CI/CD pipelines
- DevOps practices
- Modern web and mobile interfaces

The basic principle remains the same: **modernize gradually instead of replacing everything at once.**

15. Case Study – College Management System

Existing System

Suppose a college has an old desktop application containing:

Student Information

Attendance

Marks

Examination

Reports

The system works correctly but cannot easily support mobile users.

Proposed Modernization

A new mobile/web application is introduced.

Students / Teachers

|



Web / Mobile Application

|



REST API

|



Adapter / Wrapper

|



Legacy College System

|



Existing Database

Modernization Steps

Step 1: Analyze the existing system.

Step 2: Refactor complicated code.

Step 3: Develop REST APIs.

Step 4: Add an adapter where required.

Step 5: Connect the new web/mobile interface.

Step 6: Migrate selected data if required.

Step 7: Perform unit, integration and regression testing.

Step 8: Use Git for version control.

Step 9: Use CI/CD for automated build and deployment.

This approach allows the college to keep its existing system while gradually adding modern features.

16. Challenges

The main challenges are:

- Understanding undocumented legacy code
- Maintaining compatibility
- Migrating large amounts of data
- Preventing downtime
- Testing old and new components together
- Managing security
- Training developers
- Maintaining old and new technologies simultaneously

17. Future Scope

Future modernization can include:

- Complete cloud migration
- Microservice-based architecture
- Containerization using Docker

- Automated CI/CD deployment
- API gateway integration
- Improved cybersecurity
- Real-time monitoring
- Mobile-first interfaces
- Automated database migration
- Gradual replacement of obsolete components

18. Why This Topic is Important

The important idea of this project is **not simply replacing old software**.

Instead:

OLD SYSTEM

↓

Analyze

↓

Refactor

↓

API Integration

↓

Adapter / Wrapper

↓

Testing

↓

CI/CD

↓

MODERN SYSTEM

This makes modernization safer, more affordable and easier to manage.

19. Conclusion

Updating legacy software is often better than completely replacing it. Organizations can modernize their existing systems gradually using **code refactoring, API integration, adapters, database migration, automated testing, Git and CI/CD**.

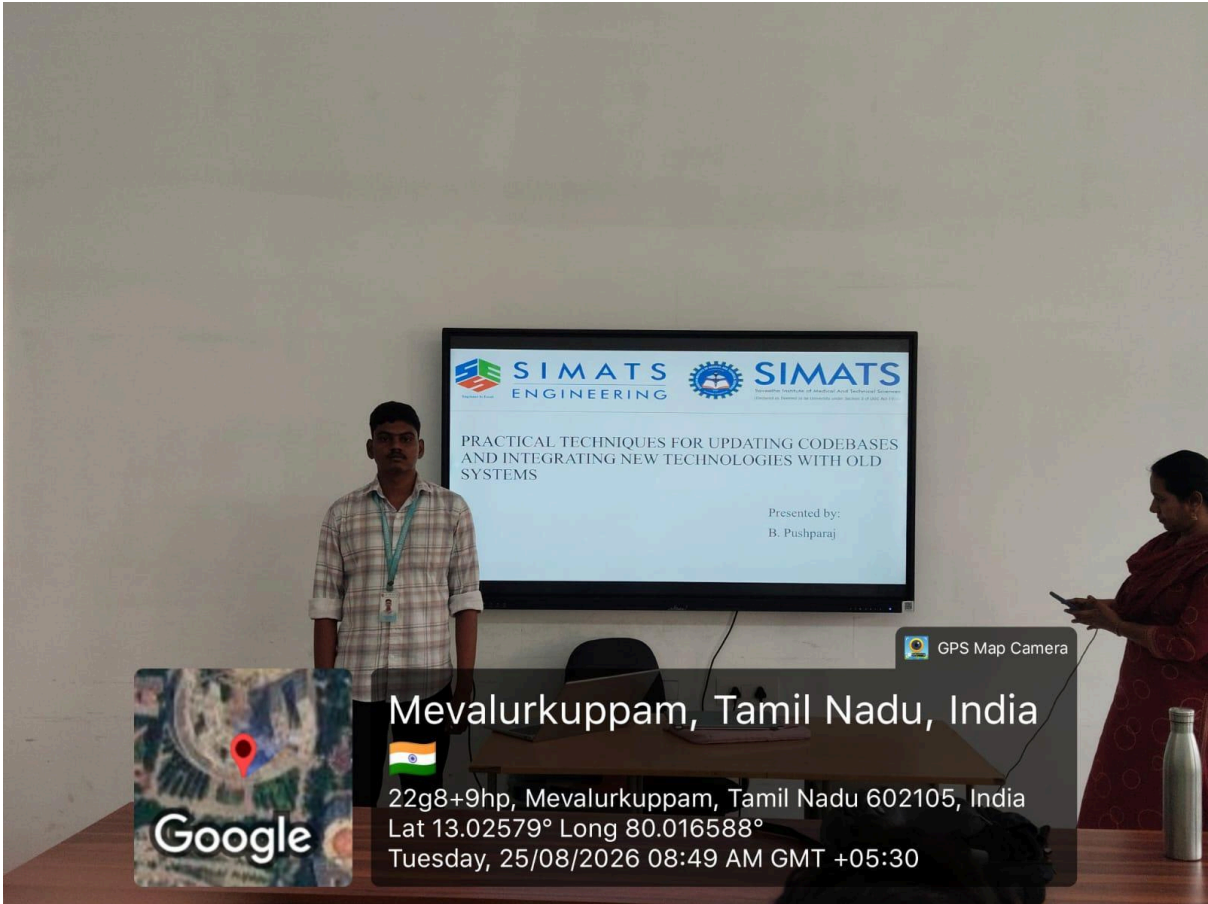
These techniques help organizations connect reliable old systems with modern technologies while reducing cost, downtime and risk.

As your PPT concludes, modernization should be performed step-by-step so that old reliable systems can be connected with new technologies safely and efficiently.

20. References

1. Your seminar PPT: **“Practical Techniques for Updating Codebases and Integrating New Technologies with Old Systems.”**
2. SIMATS Engineering, **CO5 Assessment Tool 2 – Seminar on Emerging Technologies**, Software Engineering.
3. Martin Fowler, *Refactoring: Improving the Design of Existing Code*.
4. Eric Evans, *Domain-Driven Design*.
5. Robert C. Martin, *Clean Code*.
6. Microsoft documentation on application modernization.
7. Docker documentation on application containerization.
8. Git documentation on version control and branching.
9. REST API architectural principles.
10. General software engineering practices for legacy-system modernization.

Geotag Photo of the seminar



GPS Map Camera

Mevalurkuppam, Tamil Nadu, India

22g8+9hp, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.02579° Long 80.016588°
Tuesday, 25/08/2026 08:49 AM GMT +05:30